

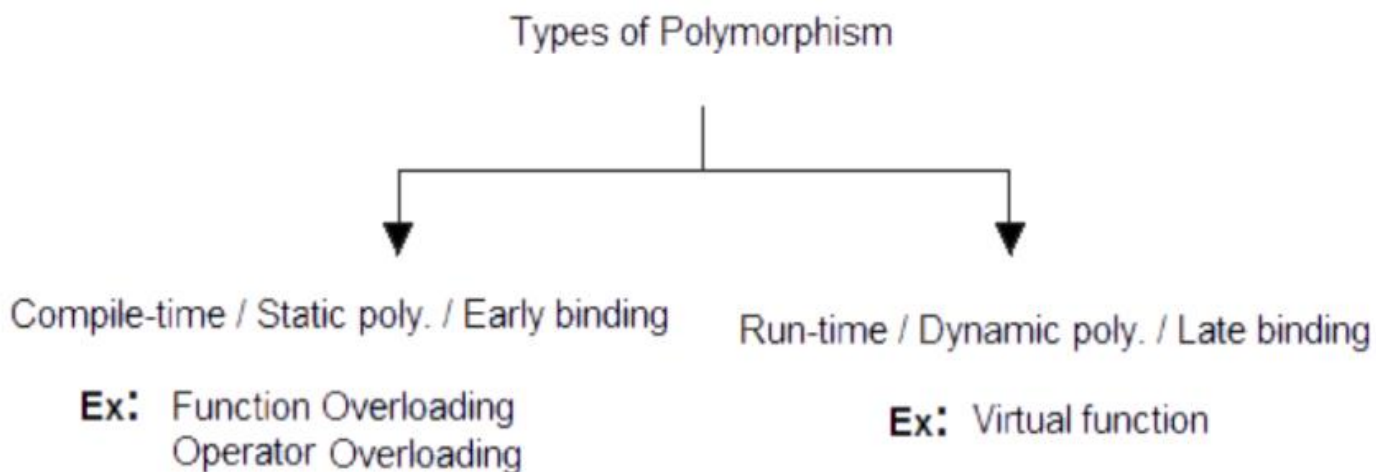
C++ Polymorphism

- Polymorphism is the combination of two greek word one is poly means many and another is morphism means form.
- Polymorphism means more than one function with same name, with different working.

Real Life Example Of Polymorphism

Let's consider a real-life example of polymorphism. A lady behaves like a teacher in a classroom, mother or daughter in a home and customer in a market. Here, a single person is behaving differently according to the situations.

There are two types of polymorphism in C++:



- **Compile time polymorphism:**
The overloaded functions are invoked by matching the type and number of arguments.
This information is available at the compile time and, therefore, compiler selects the appropriate function at the compile time.
It is achieved by function overloading and operator overloading which is also known as static binding or early binding. Memory allocation done at compile time.
- **Run time polymorphism:**
Run time polymorphism is achieved when the object's method is invoked at the run time instead of compile time.
It is achieved by method overriding.
which is also known as dynamic binding or late binding. memory allocation done at runtime.

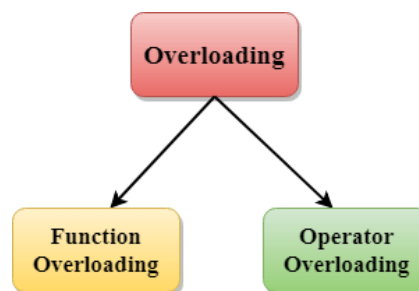
Differences b/w compile time and run time polymorphism.

Compile time polymorphism	Run time polymorphism
The function to be invoked is known at the compile time.	The function to be invoked is known at the run time.
It is also known as overloading, early binding and static binding.	It is also known as overriding, Dynamic binding and late binding.
Overloading is a compile time polymorphism where more than one method is having the same name but with the different number of parameters or the type of the parameters.	Overriding is a run time polymorphism where more than one method is having the same name, number of parameters and the type of the parameters.
It is achieved by function overloading and operator overloading.	It is achieved by virtual functions and pointers.
It provides fast execution as it is known at the compile time.	It provides slow execution as it is known at the run time.
It is less flexible as mainly all the things execute at the compile time.	It is more flexible as all the things execute at the run time.

C++ Overloading (Function and Operator)

If we create two or more members having the same name but different in number or type of parameter, it is known as C++ overloading. In C++, we can overload:

- function,
- constructors, and
- operators



Types of overloading in C++ are:

- Function overloading
- Operator overloading
- Constructor overloading

C++ Function Overloading

Function Overloading is the process of having two or more function with the same name, but different in parameters is known as function overloading in C++.

C++ Function Overloading Example

```
#include <iostream.h>

#include<conio.h>
class addition
{
    public:
    void add(int a,int b)
    {
        cout<<"a+b:"<<a+b;
    }
    void add(int a, int b, int c)
    {
        cout<<"a+b+c:"<<a+b+c;
    }
};

void main()
{
    addition a1;
    a1.add(2,3);
    a1.add(12, 20, 23);
    getch();
}
```

C++ Operators Overloading

Operator overloading is a compile-time polymorphism in which the operator is overloaded to provide the special meaning to the user-defined data type.

Operator overloading is used to overload or redefines most of the operators available in C++.

Operator that cannot be overloaded are as follows:

- Scope operator (::)
- Sizeof
- member selector(.)
- member pointer selector(*)
- ternary operator(?:)

Syntax of Operator Overloading:

(Syntax for Inside of the class)

```
return_type operator op(argument_list)
{
    // body of the function.
}
```

(Syntax for Outside of the class)

```
return_type class_name :: operator op(argument_list)
{
    // body of the function.
}
```

Where the **return type** is the type of value returned by the function.

class_name is the name of the class.

operator op is an operator function where op is the operator being overloaded, and the operator is the keyword.

Example:

```
#include <iostream.h>
#include <conio.h>
class magic
{
```

Object Oriented Concepts & Data Structure
Unit 3 Polymorphism

```
private:
    int a; public:
    magic()
    {
        a=0;
    }
    void operator ++()
    {
        a +=10;
    }
    void display()
    {
        cout<<"\nA is: "<<a;
    }
};
void main()
{
    clrscr();
    magic m1;
    m1.display();
    m1 ++;      // calling ++()
    m1.display();
    getch();
}
```

Object Oriented Concepts & Data Structure

Unit 3 Polymorphism

```
/* use binary (+) operator to perform the addition of two numbers. */
#include <iostream.h>
#include<conio.h>
class num
{
    // declare data member or variable
    int x, y;
    public:
    // create a member function to take input
    void input()
    {
        cout << " Enter the first number: ";
        cin >> x;
    }
    void input2()
    {
        cout << " Enter the second number: ";
        cin >> y;
    }
    // overloading the binary '+' operator to add number
    num operator + (num ob)
    {
        // create an object
        num A;
        // assign values to object
        A.x = x + ob.x;
        return A;
    }
    // display the result of binary + operator
    void print()
    {
        cout << "The sum of two numbers is: " <<x;
    }
};

void main ()
{
    clrscr();
    num x1, y1, res; // here we create object of the class num i.e x1 and y1
    // accepting the values
    x1.input();
    y1.input();
    // assign result of x1 and x2 to res
    res = x1 + y1;
    // call the print() function to display the results
    res.print();
    getch();
}
```

Rules for Operator Overloading

- Existing operators can only be overloaded, but the new operators cannot be overloaded.
- The overloaded operator contains at least one operand of the user-defined datatype.
- We can't change the basic meaning of operator.
- Some operators can't be overloaded: member access operator(.), scope resolution operator(::) etc.
- When unary operators are overloaded through a member function take no explicit arguments.
- When binary operators are overloaded through a member function takes one explicit argument.

C++ Function Overriding

Function Overriding is not possible in one class for it we must use two class. Generally it is perform in inheritance.

If derived class defines same function as defined in its base class, it is known as function overriding in C++.

It is used to achieve runtime polymorphism.

It enables you to provide specific implementation of the function which is already provided by its base class.

C++ Function Overriding Example

Let's see a simple example of Function overriding in C++. In this example, we are overriding the display() function.

```
#include <iostream.h>
#include <conio.h>

class magic1
{
    public:
    void display()
    {
        cout<<"\nDisplay from Magic1class...";
    }
};

class magic2: public magic1
{
    public:
    void display()
    {
        cout<<"\nDisplay from Magic2class...";
    }
};

void main()
{
    clrscr(); magic2 m2;
    m2.magic1::display(); // call magic1class method
    m2.display();
    getch();
}
```

Object Oriented Concepts & Data Structure
Unit 3 Polymorphism

Function Overloading	Function Overriding
Function overloading can be used in normal functions as well as in classes (for eg: constructor overloading is a classic example where you would vary the number/type of arguments for different initialisations)	Function overriding is applicable exclusively to an inherited class (or in other words a subclass)
Function overloading is resolved at compile time	Function overriding is resolved at run time.
Overloaded functions are in same scope	Overridden functions are in different scopes
Overloaded functions have different function signatures	Overridden functions have same function signatures

C++ virtual function

Consider a base class and derived class that contains member function which has the same name. Now, we want to access member function as per our requirement either from the base class or from the derived class at runtime.

For that we create a pointer of based class to refer to all derived objects, but we know that even when it is make to contents the address of derived class, it always execute the function in the base class.

To overcome this problem virtual function is used to achieve polymorphism at runtime

Virtual function is a Dynamic polymorphism or Late binding.

A C++ virtual function is a member function in the base class that you redefine in a derived class.

It is declared using the virtual keyword.

When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Rules of Virtual Function

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

Example: (v2.cpp)

```
#include <iostream.h>

#include<conio.h>

class A
{
public:

virtual void display()
{
cout << "Base class is invoked"<<endl;
}
};

class B : public A
{
public:

void display()
{
cout << "Derived Class is invoked"<<endl;
}
};

void main()
{
clrscr();

A *a;

A a1;
```

```
B b1;  
  
a=&a1;  
a->display();  
  
a=&b1;  
  
a->display();  
getch();  
}
```

Pure Virtual Function

- A pure virtual function is a type of function which have only function declaration . It does not have function definition.
- When the function has no definition, such function is known as "**do-nothing**" function.
- The "**do-nothing**" function is known as a **pure virtual function**. A pure virtual function is a function declared in the base class that has no definition relative to the base class.

Pure virtual function can be defined as:

virtual void getdata() = 0; **or** virtual void putdata(){}

Example:

```
#include<iostream.h>  
#include<conio.h>  
class Base  
{  
protected:  
  
int a,b;  
  
public:  
  
virtual void getdata(){}  
virtual void putdata()=0;  
  
};  
class Derived : public Base  
{  
  
int c;  
  
public:
```

Object Oriented Concepts & Data Structure
Unit 3 Polymorphism

```
void getdata()
{
    cout<<"\nEnter a :";

    cin>>a;

    cout<<"\n Enter b:";

    cin>>b;

    cout<<"\n Enter c:";

    cin>>c;
}
void putdata()
{
    cout<<"\n A is:"<<a;
    cout<<"\n B is:"<<b;
    cout<<"\n C is:"<<c;
}
};
```

```
void main()
{
    clrscr(); Base
        *b;
        Derived d;
        b=&d;
        b->getdata();
        b->putdata();
        getch();
}
```

C++ Friend function

- If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.
- By using the keyword friend compiler knows the given function is a friend function.
- For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

```
class class_name
{
    friend data_type function_name(argument/s);    // syntax of friend function.
};
```

In the above declaration, the friend function is preceded by the keyword friend.

The function can be defined anywhere in the program like a normal C++ function.

The function definition does not use either the keyword **friend** or **scope resolution operator**.

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.
- Usually, it has an object as argument.

Example of friend Function with 1 Class:

```
#include<iostream.h>
#include<conio.h>

class magic
{
    private:

        int a,b;

    public:

        void getdata();
        friend void add(magic);
};

void magic::getdata()
{
    cout<<"Enter a:";
    cin>>a;
    cout<<"\nEnter b:";
```

Object Oriented Concepts & Data Structure
Unit 3 Polymorphism

```
cin>>b;
}
void add(magic m1)
{
cout<<m1.a+m1.b;
}
void main()
{
clrscr();

magic m1;

m1.getdata();    //Normal function calling
add(m1); //Friend Function Calling
getch();
}
```

Output:

```
Enter a : 25
Enter b : 25
Addition is : 50
```

Example:2 Friend Function with 1 Class

```
#include<iostream.h>
#include<conio.h>
#include<stdio.h>

class student
{
// data member by defaultint
roll_no;
char name[20];
public:
void get_record(void); //normal member declaration
friend void show_record(student); //friend declaration in public

};
```

Object Oriented Concepts & Data Structure
Unit 3 Polymorphism

```
void student:: get_record()    //outside definition using scope resolution operator(::)
{
cout<<"Enter roll no: ";
cin>>roll_no; cout<<"Enter
Name : ";gets(name);
}
void show_record(student s2) //outside definition of friend function
                           //without scope resolution operator(::)
{
cout<<"\nRoll no: "<<s2.roll_no;// accessing first private member
cout<<"\nName : "<<s2.name;    // accessing second private member
}

void main() // main function start here
{
clrscr();

student s1;
s1.get_record(); //normal member calling
show_record(s1); //friend function calling
getch();
}
```

Output:

Enter rollno: 101
Enter Name: Tanay

Roll No:101Name:Tanay

Let's see a simple example when the function is friendly to two classes.

```
#include<iostream.h>
#include<conio.h>

class second;    //forward Declaration of class

class first
{
private:
    int a;
    friend void sum(first,second);
```

Object Oriented Concepts & Data Structure
Unit 3 Polymorphism

```
public:
    void getdata()
    {
        cout<<"Enter a:";cin>>a;
    }
};
class second
{
private:
    int b;
    friend void sum(first,second);public:
void getdata()
{
    cout<<"Enter b:";cin>>b;
}
};
void sum(first f1,second s1)
{
    cout<<"Addition is:"<<f1.a+s1.b;
}
void main()
{
    clrscr();

    first f1;
    second s1;

    f1.getdata();
    s1.getdata();

    sum(f1,s1);

    getch();
}
```

Output:

Enter a : 25

Enter b : 25

Addition is: 50